

ВВЕДЕНИЕ

В условиях современного информационного общества возникает необходимость в эффективных и масштабируемых решениях для хранения и обработки данных. Именно поэтому системы управления базами данных (СУБД) становятся важнейшим инструментом для организации информационных потоков в бизнесе, веб-приложениях и других IT-проектах. В последние годы значительно возрос интерес к нереляционным базам данных (NoSQL), которые предлагают инновационные подходы к работе с данными. Одной из самых востребованных NoSQL-систем является MongoDB – гибкая, высокопроизводительная и масштабируемая база данных, которая активно используется в различных сферах. Цель данной практической работы – познакомить студентов с ключевыми принципами и особенностями работы с MongoDB. В рамках выполнения заданий студенты освоят базовые навыки работы с этой NoSQL-базой данных, а также узнают, где и как MongoDB может быть наиболее полезной. В процессе практики будут изучены основные операции CRUD (создание, чтение, обновление, удаление), работа с индексами, выполнение агрегационных запросов, а также методы повышения производительности и масштабирования MongoDB. Кроме того, будет рассмотрено применение MongoDB в таких областях, как веб-разработка, аналитика данных, мобильные приложения и другие. Изучение MongoDB в рамках данной практики позволит студентам углубить свои знания в области баз данных и подготовиться к работе с современными технологиями обработки данных в профессиональной деятельности.

1 NoSQL база данных MongoDB

MongoDB была разработана компанией 10gen, которая позже изменила свое название на MongoDB Inc.. Основателями компании являются Дуайт Мердок, Кевин Пулс, и Элиот Горовиц. Эти люди сыграли ключевую роль в создании и развитии MongoDB как одной из самых популярных NoSQL баз данных.

Основатели MongoDB: Дуайт Мердок, Кевин Пулс, Элиот Горовиц. Дуайт Мердок один из сооснователей, который занимался архитектурой и дизайном системы. Его опыт в разработке программного обеспечения и понимание потребностей разработчиков помогли сформировать MongoDB как мощный инструмент для работы с данными. Кевин Пулс также сооснователь, который внес значительный вклад в разработку и продвижение MongoDB. Его работа сосредоточена на создании удобного интерфейса и функциональности, которые делают MongoDB привлекательной для разработчиков. Элиот Горовиц третий сооснователь, который занимался техническими аспектами разработки. Его знания в области распределенных систем и баз данных помогли создать надежную и масштабируемую платформу.

MongoDB была запущена в 2009 году и быстро завоевала популярность благодаря своей способности обрабатывать большие объемы данных и гибкости в работе с документами, что отличает ее от традиционных реляционных баз данных. Она использует JSON-подобные документы, что делает ее удобной для разработчиков, работающих с современными веб-приложениями и большими данными.

Ссылка на официальный сайт разработчиков системы:
<https://www.mongodb.com/> [1].

1.1 Описание системы

MongoDB это документоориентированная система управления базами данных, которая относится к категории NoSQL. Основное назначение MongoDB заключается в следующем:

Гибкость в структуре данных: MongoDB использует документы в формате JSON (или BSON), что позволяет хранить данные без строгой схемы. Это означает, что документы в одной коллекции могут иметь разные поля, что делает систему очень гибкой для работы с изменяющимися данными.

Масштабируемость: MongoDB разработана для работы с большими объемами данных и может легко масштабироваться горизонтально, что позволяет добавлять новые серверы для увеличения производительности и хранения данных.

Высокая производительность: Благодаря своей архитектуре, MongoDB обеспечивает быструю обработку запросов и эффективное управление данными, что делает ее подходящей для приложений с высокими требованиями к производительности.

Поддержка сложных запросов: MongoDB поддерживает вторичные индексы, агрегацию и запросы с диапазонами, что позволяет выполнять сложные операции над данными.

Использование в различных приложениях: MongoDB подходит для различных типов приложений, включая веб-приложения, мобильные приложения, системы управления контентом и аналитические платформы, благодаря своей универсальности и способности обрабатывать разнообразные типы данных.

MongoDB имеет уникальную архитектуру, которая отличается от традиционных реляционных баз данных. Основные компоненты и особенности архитектуры MongoDB включают:

- 1) **Документоориентированная модель данных:** В MongoDB данные хранятся в виде документов, которые представляют собой структуры, подобные

JSON (BSON). Это позволяет хранить сложные и вложенные данные, что делает систему более гибкой по сравнению с реляционными таблицами.

Коллекции: Документы организованы в коллекции, которые аналогичны таблицам в реляционных базах данных. Каждая коллекция может содержать документы с различной структурой, что позволяет легко адаптироваться к изменениям в данных.

Репликация: Для обеспечения надежности и отказоустойчивости MongoDB использует репликацию. Это означает, что данные могут быть дублированы на нескольких серверах, что позволяет восстанавливать данные в случае сбоя одного из узлов.

Агрегация: MongoDB предоставляет мощный механизм агрегации, который позволяет выполнять сложные запросы и обрабатывать данные на лету. Это делает возможным извлечение и анализ данных без необходимости предварительной обработки.

Интерфейс для запросов: MongoDB использует собственный язык запросов, который позволяет разработчикам легко взаимодействовать с базой данных. Запросы могут включать фильтрацию, сортировку и проекцию данных, что делает работу с данными интуитивно понятной.

Шардирование: MongoDB поддерживает горизонтальное масштабирование через шардирование, что позволяет распределять данные по нескольким серверам (шардам). Это обеспечивает высокую доступность и производительность при работе с большими объемами данных.

1.2 Репликация

Репликация в MongoDB — это механизм, который автоматически создает и поддерживает резервные копии данных, дублируя их на нескольких серверах. Основная цель этого процесса — обеспечение надежности и устойчивости базы данных. Рассмотрим ключевые особенности и преимущества репликации в MongoDB:

1) Отказоустойчивость: в случае сбоя одного из серверов, система автоматически переключается на другой, что гарантирует непрерывную работу.

2) Шкалируемость: при увеличении нагрузки на базу данных репликация позволяет распределять запросы между несколькими репликами, что способствует улучшению производительности

3) Резервное копирование: каждая реплика содержит полную копию данных, что позволяет восстановить базу в случае повреждения или потери информации.

4) Чтение данных: дополнительные реплики могут использоваться для чтения, что увеличивает общую пропускную способность системы.

Основные элементы репликации в MongoDB включают основной узел (Primary), который принимает записи данных, и несколько вторичных узлов (Secondary), которые дублируют информацию с основного узла. Этот механизм автоматически обнаруживает сбой и переключается на доступные реплики, обеспечивая тем самым непрерывность работы системы.

1.3 Шардирование

Шардирование в MongoDB — это метод, который позволяет разделять данные и распределять их по нескольким серверам, известным как шардов, с целью повышения масштабируемости и производительности базы данных. Эта функция позволяет эффективно обрабатывать большие объемы информации и улучшает скорость выполнения запросов.

Процесс шардирования автоматически распределяет данные по различным шардам на основе заданных ключей, что помогает балансировать нагрузку и обеспечивает равномерное распределение данных.

Для настройки шардирования в MongoDB необходимо определить несколько ключевых элементов, таких как ключи шардирования, сами шардов, маршрутизатор запросов (mongos) и конфигурационный сервер. Эта система

также поддерживает горизонтальное масштабирование, позволяя добавлять новые шарды по мере увеличения нагрузки и объема данных.

В целом, шардирование в MongoDB обеспечивает эффективное распределение данных между серверами, что способствует повышению производительности, масштабируемости и надежности базы данных.

1.4 Преимущества и недостатки системы MongoDB

Преимущества MongoDB:

1) Гибкость схемы (Schema-less) MongoDB не требует жесткой схемы данных, что позволяет легко изменять структуру данных без необходимости миграции базы. Это особенно полезно для приложений с динамически изменяющимися требованиями.

2) Масштабируемость. MongoDB поддерживает горизонтальное масштабирование с помощью шардинга (sharding), что позволяет эффективно работать с большими объемами данных. При увеличении объема данных система автоматически распределяет их между узлами без остановки работы.

3) Высокая производительность. Благодаря документно-ориентированной модели и использованию кэша и оперативной памяти, MongoDB обеспечивает высокую скорость операций чтения и записи по сравнению с традиционными реляционными базами данных.

4) Поддержка геопространственных данных. MongoDB предоставляет встроенные возможности для работы с геопространственными данными, что делает её удобной для приложений, связанных с картами и геолокацией.

5) Богатый язык запросов. MongoDB предлагает мощный и выразительный язык запросов, который поддерживает фильтрацию, сортировку, агрегацию и другие сложные операции.

6) Высокая доступность. Использование репликации (replica sets) обеспечивает отказоустойчивость и высокую доступность данных.

Недостатки MongoDB:

1) Высокое потребление памяти. MongoDB использует формат BSON (Binary JSON), который занимает больше памяти по сравнению с традиционными реляционными базами данных. Это может стать проблемой при работе с большими объемами данных.

2) Ограничения в сложных транзакциях. MongoDB не поддерживает сложные транзакции на уровне, сравнимом с реляционными базами данных. Это может быть недостатком для приложений, требующих строгой согласованности данных.

3) Проблемы с согласованностью (Eventual Consistency). В некоторых сценариях MongoDB может обеспечивать только "в конечном итоге согласованную" модель данных, что не всегда подходит для приложений, где требуется строгая согласованность.

4) Отсутствие поддержки сложных реляционных связей. MongoDB не предназначена для работы с данными, имеющими сложные реляционные связи, что делает её менее подходящей для некоторых типов приложений.

5) Проблемы с производительностью при больших объемах данных. При работе с очень большими наборами данных MongoDB может потребовать значительных ресурсов для поддержания производительности.

6) Ограниченная поддержка ACID. Хотя MongoDB поддерживает транзакции, её возможности в этой области уступают реляционным базам данных, что может быть критичным для финансовых или банковских приложений.

2 Установка MongoDB

1) Перейти на официальный сайт разработчиков (<https://www.mongodb.com/try/download/community>) и скачать файл с расширением msi (см. рис. 1). При помощи этого установщика мы поставим на компьютер сервер базы данных MongoDB, а также инструмент для работы с ним Compass.

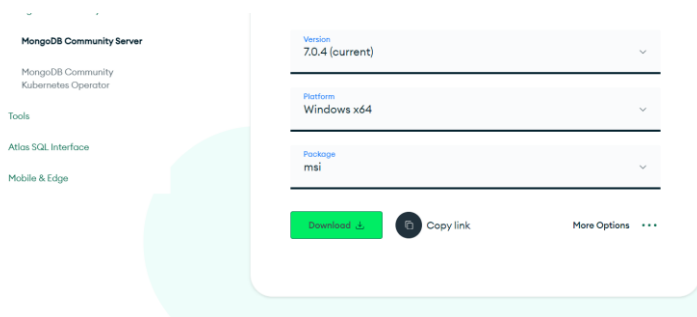


Рисунок 1 Скачивание загрузочного пакета

2) Запускаем установку NoSQL системы MongoDB при помощи загруженного пакета (см. рис. 2).



Рисунок 2 Окно установки MongoDB

3) Выбираем Complete, чтобы установить все необходимые инструменты для работы с MongoDB (см. рис. 3).

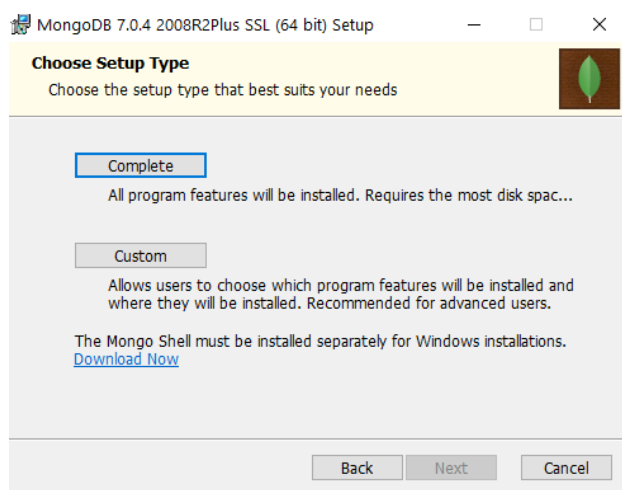


Рисунок 3 Окно выбора режима установки

4) В окне настроек службы MongoDB ничего не меняем, оставляем стандартные настройки (см. рис. 4).

Добавлено примечание ([N_1]): .

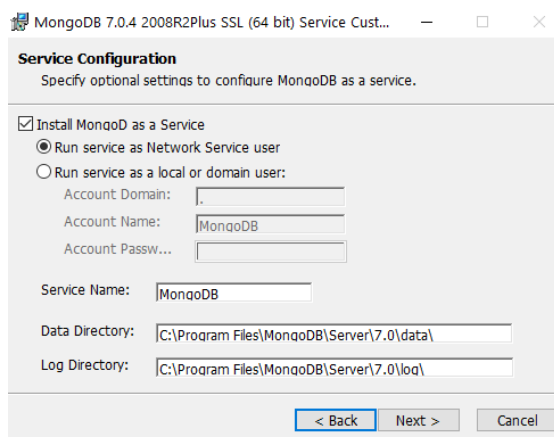


Рисунок 4 Окно настроек службы MongoDB

5) Устанавливаем также MongoDB Compass – это удобный инструмент для работы с базами данных MongoDB. Для этого можно поставить галочку в установщике (см. рис. 5).

Добавлено примечание ([N_2]):

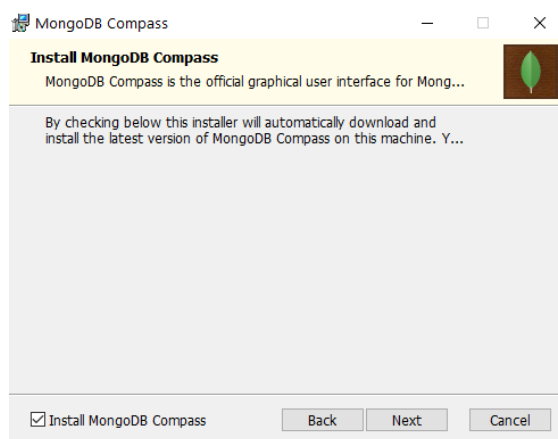


Рисунок 5 Окно выбора установки MongoDB Compass

6) После этого начнётся процесс установки приложения (см. рис. 6). Установка займёт некоторое время, стоит подождать.

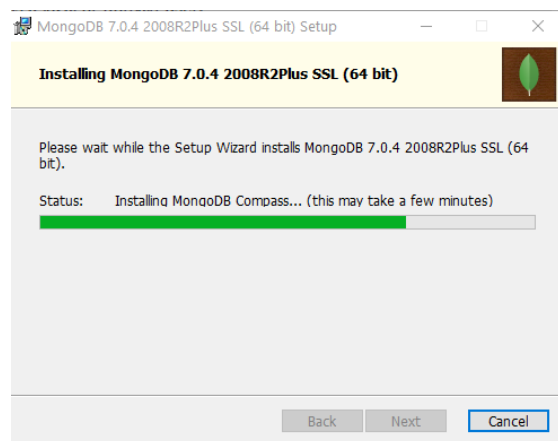


Рисунок 6 Процесс установки

7) После запуска приложения MongoDB Compass нажмите на кнопку Start (см. рис. 7).

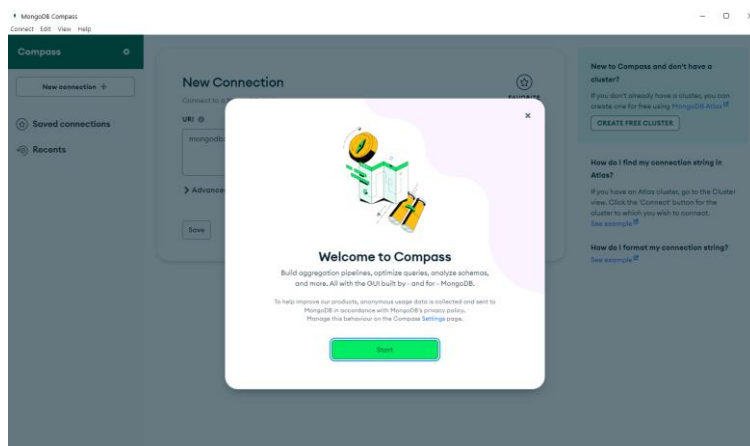


Рисунок 7 Окно приложения при первом запуске

8) После нужно соединиться с базой данных при помощи кнопки «Connect» (см. рис. 8).

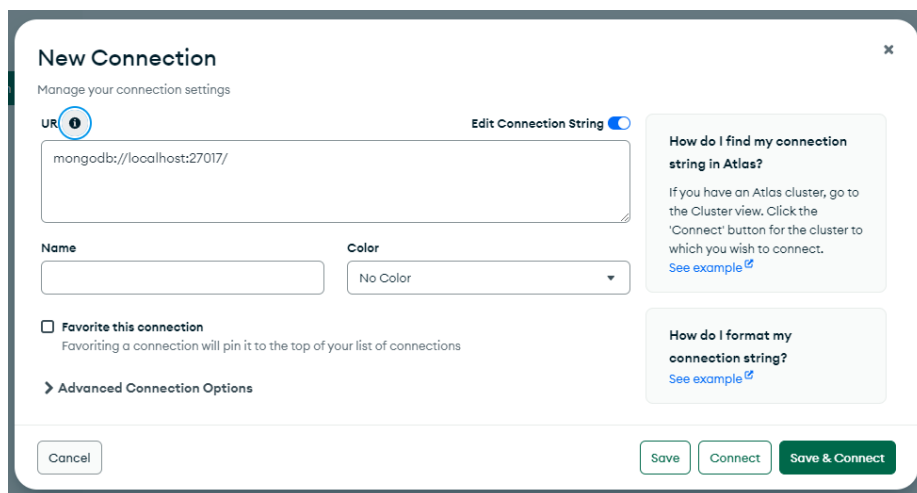


Рисунок 8 Создание соединения с сервисом MongoDB

9) Чтобы добавить новую базу данных нужно нажать на кнопку с плюсом, или на кнопку «Create database» (см. рис. 9).

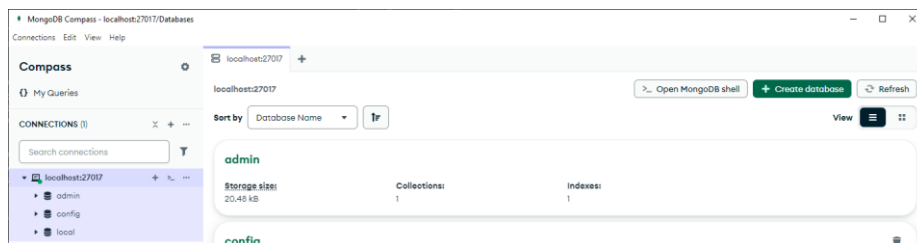


Рисунок 9 Главное меню MongoDB Compass. Список баз данных

3 Сведения о Датасете

Название датасета «Spotify Tracks»

Содержание: набор данных включает в себя сведения о песнях на музыкальном сервисе Spotify.

Кол-во строк: 62318.

Поля датасета представляют из себя данные песни, например, название, исполнитель, год издани и другие. Список всех полей можно просмотреть в таблице 3.1.

Таблица 3.1 Поля датасета «Spotify Tracks»

№	Поле	Описание
1	track_id	Spotify-идентификатор трека
2	track_name	Название трека
3	artist_name	Имена исполнителей, которые исполняли трек, разделяются запятыми, если их несколько
4	year	Год выпуска трека
5	popularity	Значение от 0 до 100, указывающее, насколько популярен трек в зависимости от количества прослушиваний и новизны
6	artwork_url	URL-адрес альбома или обложки трека
7	album_name	Альбом, в котором появляется трек
8	acousticness	Доверительный показатель от 0,0 до 1,0 того, является ли трек акустическим
9	danceability	Показатель того, насколько трек подходит для танцев (0,0 = наименее танцевальный, 1,0 = наиболее танцевальный)
10	duration_ms	Длина трека в миллисекундах. энергия: воспринимаемый показатель интенсивности и активности в диапазоне от 0,0 до 1,0

11	energy	Воспринимаемый показатель интенсивности и активности в диапазоне от 0,0 до 1,0.
12	key	Музыкальная тональность трека, в которой используется стандартное обозначение класса высоты звука (например, 0 = C, 1 = C#/D♭)
13	liveness	Показатель вероятности того, что трек был записан вживую (более высокие значения указывают на живое исполнение)
14	loudness	Общая громкость трека в децибелах (дБ)
15	mode	Указывает на модальность трека (1 = мажорный, 0 = минорный)
16	speechiness	Определяет наличие произносимых слов в треке (ближе к 1,0 указывает на то, что контент больше похож на речь)
17	tempo	Предполагаемый темп трека в ударах в минуту (BPM)
18	time_signature	Количество тактов в такте в диапазоне от 3 до 7
19	valence	Показатель от 0,0 до 1,0, указывающий на музыкальную позитивность трека (чем выше значение, тем лучше)
20	track_url	URL-адрес трека на Spotify
21	language	Язык трека (английский, тамильский, хинди, телугу, малаялам, корейский)

Датасет был взят с ресурса:

<https://www.kaggle.com/datasets/gauthamvijayaraj/spotify-tracks-dataset-updated-every-week?resource=download>

4 Работа с датасетом в MongoDB

После успешной установки и запуска MongoDB и MongoDB Compass пришло время работы с ним.

- 1) Запускаем приложение MongoDB Compass.
- 2) Нажать на кнопку Connect, чтобы инициализировать подключение к сервису MongoDB (см. рис. 10).

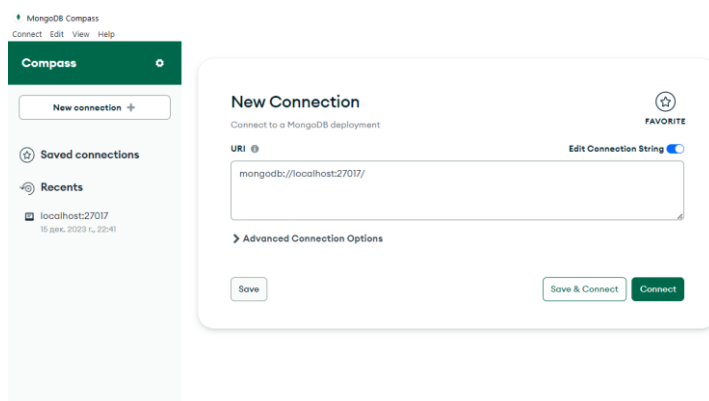


Рисунок 10 Главное меню MongoDB Compass

- 3) Далее нужно создать новую базу данных, для этого нужно нажать на кнопку с изображением плюса в меню databases (см. рис. 11).

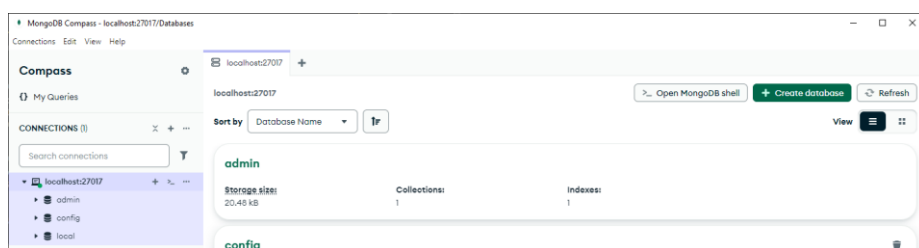
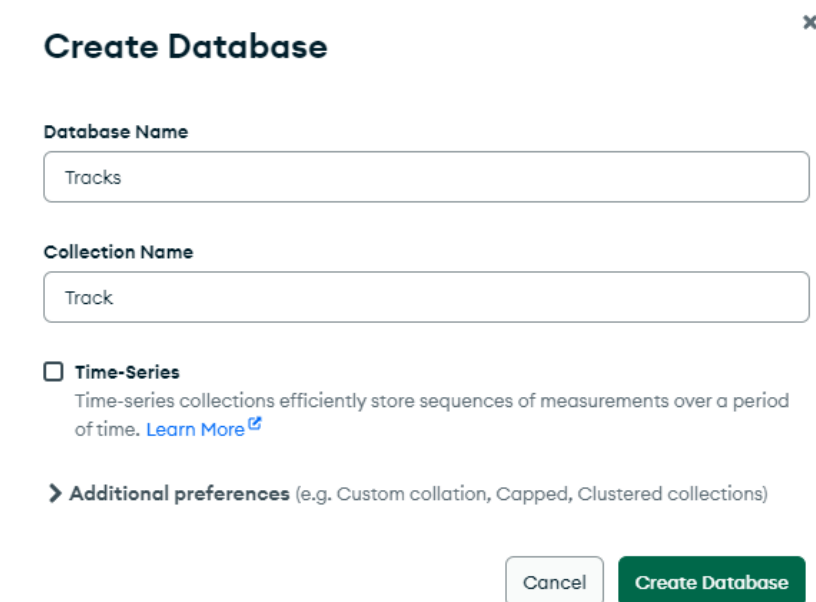


Рисунок 11 Меню databases

- 4) Написать название базы данных, для данного датасета предлагаю назвать её «Streets». Далее нужно написать название коллекции, например, «Street» (см. рис. 12).



Create Database ✕

Database Name

Tracks

Collection Name

Track

☐ **Time-Series**
Time-series collections efficiently store sequences of measurements over a period of time. [Learn More](#)

[Additional preferences](#) (e.g. Custom collation, Capped, Clustered collections)

Cancel Create Database

Рисунок 12 Создание базы данных и коллекции

5) После того, как база данных будет создана, будет видна надпись – «This collection has no data». Это означает, что коллекция пустая и её нужно наполнить данными, для этого нужно нажать на кнопку «Import Data» и вставить csv-файл датасета (см. рис. 13).

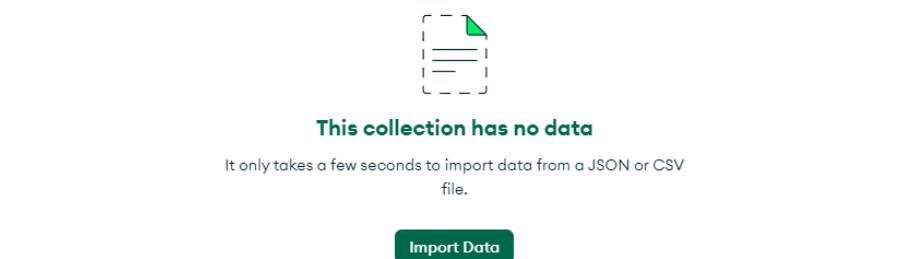


Рисунок 13 Сообщение об отсутствии данных

6) После того, как загрузится csv-файл и, если он правильно оформлен, то перед пользователем будет такое окно (см. рис. 14). Здесь предлагается поставить типы данных для различных полей коллекции, а также просмотр предварительных данных.

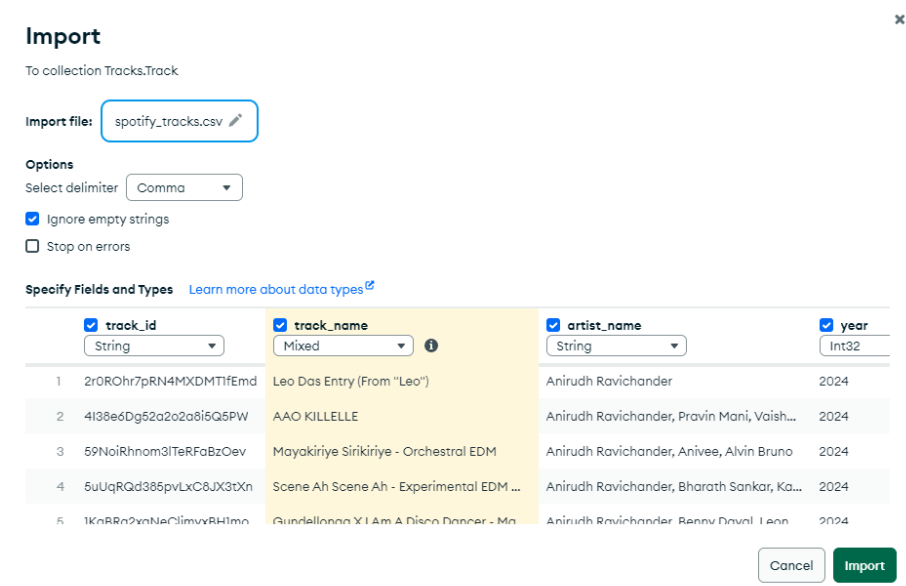


Рисунок 14 Добавление данных в коллекцию

7) Далее следует попробовать работу с конвейером в MongoDB. Конвейер нужен для того, чтобы последовательно изменять и обрабатывать данные, передавая на следующий этап новообработанные данные. Например, первым этапом отсортировали список коллекций, на следующем этапе он уже будет отсортированным и нам не нужно сортировать его заново, теперь мы группируем данные по какому-то признаку и эти группы данных перейдут на следующий этап. Сделаем группировку данных по длительности видео в датасете. Чтобы это сделать перейдем во вкладку «Aggregations» (см. рис. 15).

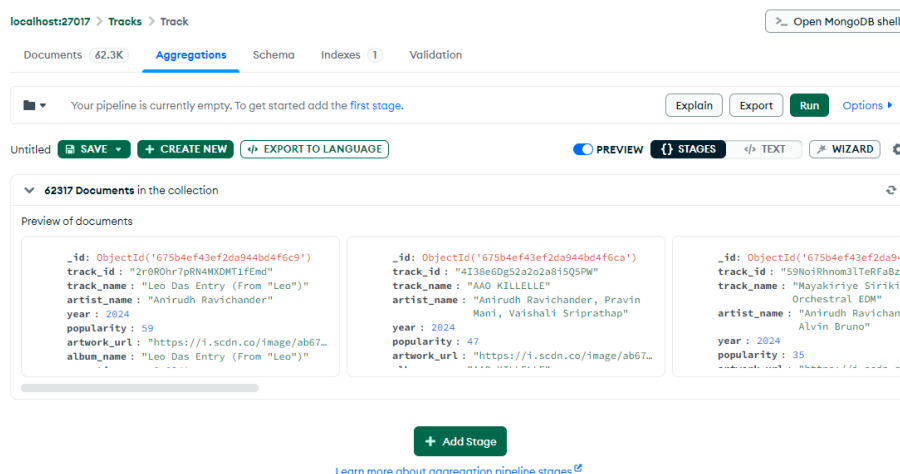


Рисунок 15 Вкладка Aggregations

8) Чтобы добавить действие на текущем этапе конвейера, нужно нажать на кнопку «Add Stage». После этого нужно выбрать действие. Чтобы отсортировать документы по какому-то признаку потребуется выбрать \$match (см. рис. 16). Отсортируем объекты по исполнителю.

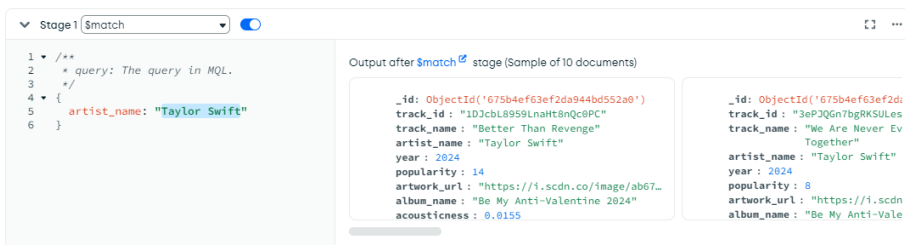


Рисунок 16 Добавление новой стадии конвейера

9) Выведем полученные данные в новую коллекцию, которую назовём «Taylor Swift». Там будут храниться документы, которые относятся к песням Тейлор Свифт (см. рис. 17).

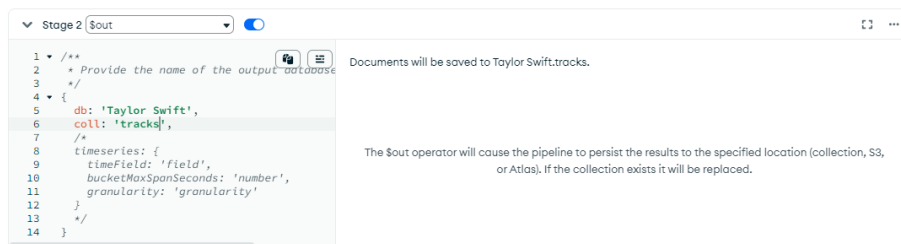


Рисунок 17 Вывод документов из контейнера в другую коллекцию

10) Проанализируем коллекцию и посмотреть статистику: когда были добавлены данные в коллекцию, частоту появления какого-то значения в документах коллекции и другие характеристики (см. рис. 19). Чтобы проанализировать коллекцию нужно перейти на вкладку «Schema» и нажать на кнопку «Analyze Schema» (см. рис. 18).

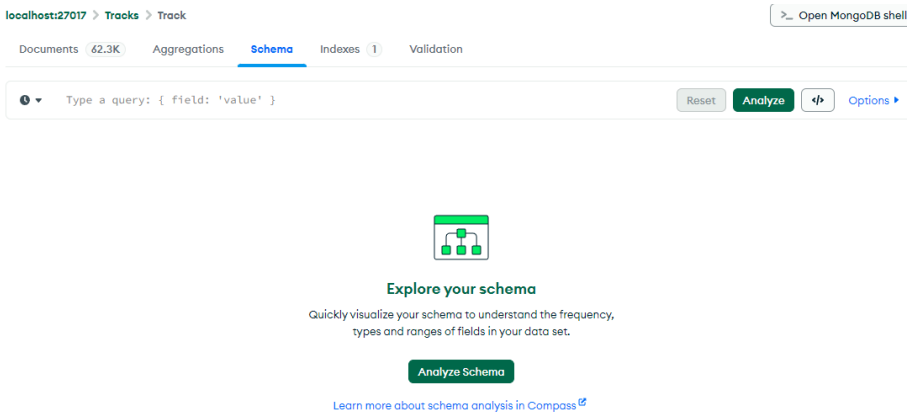


Рисунок 18 Вкладка Schema

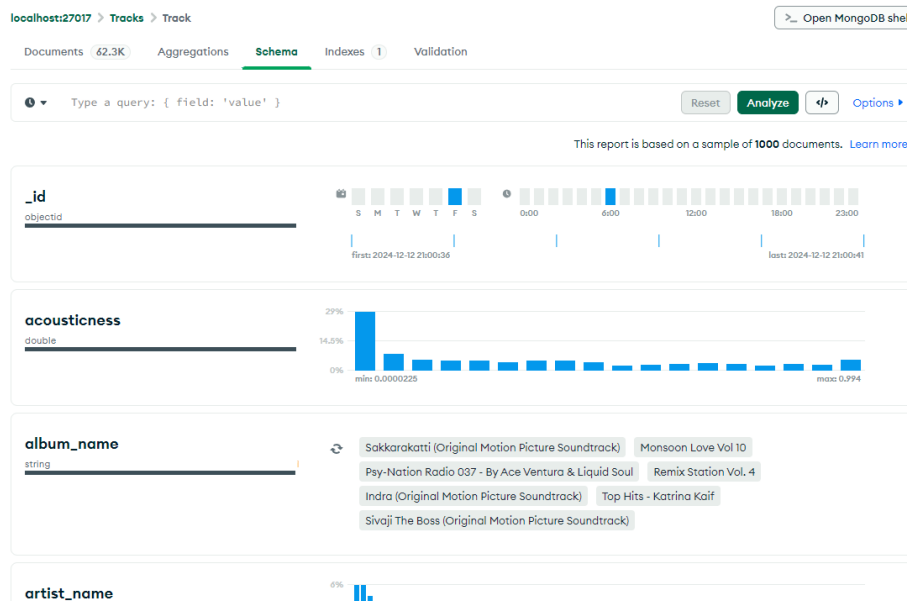


Рисунок 19 Анализ коллекции Tracks

11) Так как данных в коллекции много, то поиск по определенным параметрам может занимать большое количество времени. Чтобы ускорить поиск мы можем создавать индексы для полей, тем самым упростив поиск для

базы данных. Чтобы создать индекс нужно перейти на вкладку «Indexes» (см. рис. 20).

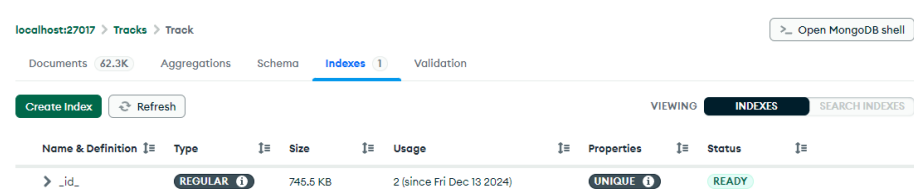


Рисунок 20 Вкладка Indexes

12) Чтобы создать индекс нужно нажать на кнопку «Create Index» и выбрать поле коллекции, для которого нужно создать индекс, а также тип поиска по индексу. Для примера создадим индекс для поля «artist_name» и тип по возрастанию (см. рис. 21).

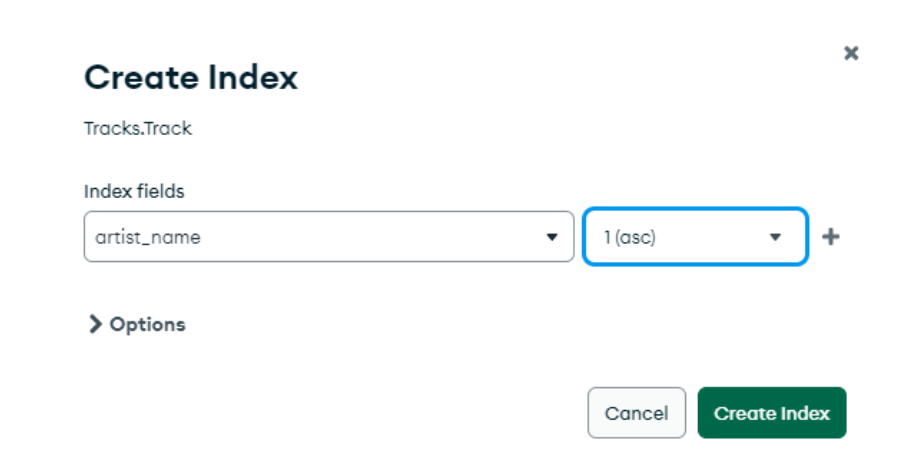


Рисунок 20 Создание индекса для поля city

13) В MongoDB можно настроить валидацию, например выбрать обязательные поля и настроить тип данных для каждого поля, кроме этого, выбрать действие при ошибке валидации, например, выдать пользователю ошибку или предупреждение. Для создания валидации нужно перейти на

вкладку «Validation» и написать BSON-объект, указав свойства `required`, `bsonType` и `properties`. Для примера создадим валидацию, где укажем, что поле `artist_name` обязательное и его тип обязательно должен быть строковым (см. рис. 22).

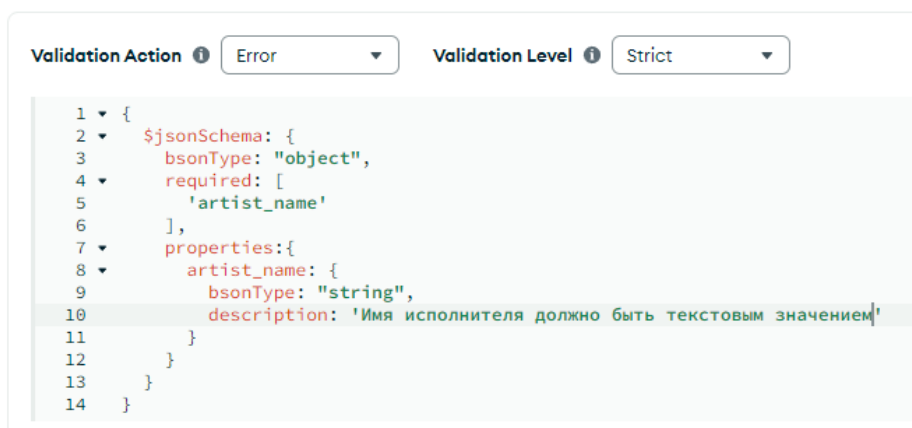


Рисунок 21 Создание валидации данных в коллекции

14) Теперь, если попробовать добавить новый документ в коллекцию и указать неверные данные, то MongoDB выдаст ошибку и мы не добавим данные (см. рис. 23).

Insert Document



To collection Tracks.Track

VIEW



```
1  /**
2  * Paste one or more documents here
3  */
4  {
5    "_id": {
6      "$oid": "675b540f3ef2da944bd5ea37"
7    },
8    "artist_name": 2
9  }
```



 Document failed validation

Cancel

Insert

Рисунок 22 Ошибка валидации при вставке неправильных данных

ЗАКЛЮЧЕНИЕ

В ходе проделанной работы были приобретены навыки работы с NoSQL системы MongoDB. Были проделаны различные действия с датасетом «Spotify Tracks».

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. MongoDB. – Текст: электронный // MongoDB, Inc. – 2024. – URL: <https://www.mongodb.com/> (дата обращения 30.11.2024).
2. MongoDB — базовые возможности. – Текст: электронный // Хабр. – 2020. – URL: <https://habr.com/ru/articles/523182/> (дата обращения 30.11.2024).